

LAB 2 — FILE UPLOAD AND COMMAND EXECUTION SECURITY ASSESSMENT

Student: Jamiu Abidemi Afolabi

Reg No: C11/26/EHIT/17319

Course: CIP-A104

Lab: Lab 2 of 5

Date: 29/08/2026

1. INTRODUCTION, SCOPE AND METHODOLOGY

1.1 Assessment Overview

This assessment focused on testing the file upload functionality of two intentionally vulnerable web applications: DVWA (Damn Vulnerable Web Application) and Mutillidae II. The aim was to identify how each application handled file uploads and document the findings.

1.2 Scope and Rules of Engagement

Testing was limited to the assigned Kali Linux lab environment, DVWA, and Mutillidae II. All testing was performed within the authorised lab, with no testing of external systems.

1.3 Laboratory Environment

The assessment was carried out using a Kali Linux virtual machine running the vulnerable web applications through docker. The applications were accessed through the local lab network and tested using standard web security tools.

1.4 Testing Methodology

Testing was performed systematically by first confirming that the applications were accessible, then reviewing their file upload functionality. Different upload scenarios were tested and the application responses were observed and documented as evidence

1.5 Test Files and Hashes

In this section, harmless test markers are created specifically for the file-upload assessment. The markers are used to identify and track uploaded content during testing. They contain no executable code, malware, external callbacks, or persistence mechanisms and are used solely within the authorised laboratory environment.

File	Extension	MIME/Content Type	Size	SHA-256
lab2-marker.txt	.txt	text/plain	25 bytes	f074db25e1f60353354b98970a d32b234d50d80fc611a177541e f44e1b92928a

lab2-marker.jpg	.jpg	image/jpeg	31,140 bytes	679e180f69354c5f92fb79dc28e be9fa26a806b193f76e1b82cd6 286c2f54237
lab2-mismatch.jpg	.jpg	text/plain	38 bytes	77bc5f316accbbabd9c9e848a0 7c8f70aa655650487d5fdef7f62 51587f2d989

2. DVWA SECURITY ASSESSMENT

2.1 File Upload Assessment

Normal Upload

Test performed:

The harmless marker named lab2-marker.jpg was uploaded through the DVWA File Upload functionality with the security level set to Low.

Observed behaviour:

DVWA accepted the uploaded file and gave a reply as follow

`../../../../hackable/uploads/lab2-marker.jpg successfully uploaded!`

Stored filename/path:

The application reported the upload path as `../../../../hackable/uploads/lab2-marker.jpg`

Retrieval behaviour:

Even though the web application didn't provide a retrieval link. The uploaded image was successfully retrieved by requesting `http://127.0.0.1:4280/hackable/uploads/lab2-marker.jpg`. No authentication was required for the retrieval within the local lab.

Evidence:

E-002 — DVWA normal upload screenshot.

Filename/Content Mismatch Test

Test performed:

For this test, a harmless text file containing the file name lab2-mismatch.jpg was uploaded to the DVWA File Upload function. The file was uploaded successfully. The file had a .jpg extension but the actual content was identified to be text/plain.

Observed behaviour:

DVWA accepted the uploaded file and reported:

`../../hackable/uploads/lab2-mismatch.jpg successfully uploaded!`

Server-side validation conclusion:

The server side accepted the filename/content mismatch. This means that the upload control doesn't reject a file based on the file's actual content type.

Evidence:

E-004 — Mismatch marker properties.

E-005 — DVWA mismatch upload response.

E-006 — DVWA mismatch retrieval result.

Finding

Observed impact:

The application accepted a file whose extension did not correspond to its actual content and stored it under the supplied .jpg filename. The file was subsequently retrievable, although the browser could not interpret it as a valid JPEG.

Potential impact:

Weak file-type validation can allow an attacker to upload content that doesn't match the expected file type. Depending on how this file are stored and served, it can lead to a more critical form of weakness where the executable content is permitted and executed by the server. Could lead to privilege escalation.

2.2 Command Injection Assessment

Normal Command Baseline

Test performed:

The initial input was modified by adding a harmless command separator and fixed marker command as follows: `127.0.0.1; echo ICDFA-LAB2-CMD-MARKER`

Observed behaviour:

The application returned the expected ping results for 127.0.0.1. It also went further to execute and return the text **ICDFA-LAB2-CMD-MARKER**. This demonstrated that the appended command was executed by the backend.

Server-side validation conclusion:

Evidence:

E-008 — DVWA command-execution proof screenshot.

Finding

Observed impact:

The DVWA Command Injection function executed the additional shell command input supplied with the IP address parameter.

Potential impact:

If a bad actor can use the same input with more capable commands, it can lead to unauthorized actions within the privilege of the application process. It might even lead to privilege escalation using reverse shell scripts.

3. OWASP MUTILLIDAE II SECURITY ASSESSMENT

3.1 File Upload Assessment

Normal Upload

Test performed:

The harmless marker jpg file was also uploaded via the Mutillidae II's Unrestricted File Upload functionality, and it was successfully uploaded.

Observed behaviour:

The application reported that the file successfully uploaded to a temporary location **/tmp/php2cbor1k34k1m0iX081N** and then moved to a permanent location with the filename **/tmp/lab2-marker.jpg**.

The application reported:

- Original filename: **lab2-marker.jpg**
- Temporary filename: **/tmp/php2cbor1k34k1m0iX081N**
- Permanent filename: **/tmp/lab2-marker.jpg**
- File type: **text/plain**

- File size: **25 Bytes**

Stored filename/path:

The application reported the upload path as `/tmp/lab2-marker.jpg`. We also verified that the file actually existed at this location with `www-data:www-data` ownership and `-rw-r--r--` permissions.

Retrieval behaviour:

No clickable retrieval link was provided by the application. The reported `/tmp/lab2-marker.jpg` location was verified inside the `www` container. However, a browser-based retrieval was not demonstrate

Evidence:

E-009 — Mutillidae normal upload screenshot.

E-010 — Mutillidae storage verification.

Finding**Observed impact:**

The application stored the uploaded file under its original filename in `/tmp/`. The SHA-256 hash matched the original marker, confirming that the stored file was unchanged.

Potential impact:

Storing uploaded content in a temporary filesystem location without strict content validation may increase risk if other application functionality can access or process the uploaded file.

Filename/Content Mismatch Test**Test performed:**

Similar to the DVWA, we also tested the Matillidae II web application for mismatch. Here, we try to upload a harmless text file containing the file name `lab2-mismatch.jpg` via the same File Upload function. The file was uploaded successfully. The file had a `.jpg` extension but the actual content was identified to be `text/plain`.

Observed behaviour:

The Mutillidae web application accepted the file and moved it to `/tmp/lab2-mismatch.jpg`. Also, the application reported that no validation was performed.

The application reported the following:

The application reported:

- Original filename: **lab2-mismatch.jpg**
- Temporary filename: **/tmp/phpdae30rp541187w8pzIk**
- Permanent filename: **/tmp/lab2-mismatch.jpg**
- File type: **image/jpeg**
- File size: **38 Bytes**

Server-side validation conclusion:

The server side accepted the filename/content mismatch, and recorded the file as image/jpeg. This means that the upload control doesn't reject a file based on the file's actual content type.

Evidence:

E-011 — Mutillidae mismatch upload screenshot.

3.2 Command Execution Assessment

Normal Command Baseline

Test performed:

Here, we tested the DNS Lookup under the Command Injection functionality of the Mutillidae II web application. We tested with a normal input: Localhost, and the app displayed the following:

Server: 127.0.0.11

Address: 127.0.0.11#53

Name: localhost

Address: 127.0.0.1

Name: localhost

Address: ::1

Observed behaviour:

The application identified the input and returned the DNS lookup information for localhost. The response displayed the DNS server as **127.0.0.1** and tied localhost to **127.0.0.1** and **::1**.

Evidence:

E-012 — Mutillidae command baseline screenshot.

Minimal Execution Proof

Now, we want to conduct the minimal execution proof to indicate the application is vulnerable to command injection. However, we will only be echoing some text to the screen without running any command that will infiltrate the app.

Test performed:

To carry out this lab, we added an echo command to the normal input we used earlier to see if the application will execute it as a command or just another text. Below is the command we injected.

localhost; echo ICDFA-LAB2-MUT-CMD-MARKER

Affected parameter:

The hostname/IP parameter field used by the DNS lookup function.

Observed behaviour/effect:

The application received the input and returned the normal DNS lookup result for localhost. The application also executed the echo command and returned the string passed to it.

Execution context:

The command was executed within the Mutillidae II application server's side environment. No privilege, unauthorised access or external connection was attempted.

Evidence:

E-013-Mutillidae-command-execution-proof.png

Finding**Observed impact:****Potential impact:**

4. COMPARATIVE ANALYSIS AND SECURE-MODE RETEST

4.1 DVWA and Mutillidae Comparison

Assessment Area	DVWA	Mutillidae II
Upload validation	At low level, the mismatched filename/content was accepted. But the same was rejected when the security level was set to "Impossible"	At security level 0, the mismatched filename/content was accepted.
Filename handling	The uploaded mismatch retained the .jpg filename.	The uploaded files retained their original filenames, including lab2-mismatch.jpg , and were moved to /tmp/ .
Storage/retrieval	The valid JPG file was stored under the application's upload path and was successfully retrieved via the URL.	While the valid JPG file wasn't able to be retrieved via the URL, it was successfully confirmed via the container that it was indeed uploaded and moved to the /tmp/ folder.

Command execution	The harmless marker attached to the IP address was executed at low but was rejected at the Impossible security level.	The DNS Lookup function executed the marker attached when the security level was 0 but rejected it as security level 1 and 5.
Execution context	The execution occurred at the application's server side command processing functionality.	The execution occurred at the application's server side DNS lookup functionality within the Mutillidae container

4.2 Observed vs Potential Impact

Finding	Observed Impact	Potential Impact
File Upload Validation	DVWA Low and Mutillidae accepted a file whose .jpg extension did not match its actual plain-text content. Mutillidae explicitly reported that validation was not performed.	Weak upload validation could allow inappropriate or malicious content to be stored or processed. In configurations where uploaded files are executable, this could potentially lead to server-side code execution.
Command Injection Validation	Both applications executed the harmless command marker supplied as part of the user input.	An attacker could potentially manipulate backend commands to perform unauthorised operations, depending on the privileges and accessible resources of the application process.

4.3 Secure-Mode Retest

Initial security level/mode:

- DVWA - Low
- Mutillidae - 0

Stronger security level/mode:

- DVWA - Impossible
- Mutillidae - 5

File upload retest result:

The same lab2-mismatch.jpg file accepted at Low was rejected at Impossible with the message:

Your image was not uploaded. We can only accept JPEG or PNG images.

The OWASP's Mutillidae II web application accepted the same mismatch markers at security level 5.

Command execution retest result:

The same harmless command-injection input that produced the marker at Low was rejected at DVWA Impossible level with: **ERROR: You have entered an invalid IP.**

But for Mutillidae II, the response was:

Malicious characters are not allowed.

Don't listen to security people. Everyone knows if we just filter dangerous characters, injection is not possible.

We use JavaScript defenses combined with filtering technology.

Both are such great defenses that you are stopped in your tracks.

Test	Weaker Mode	Stronger Mode
File upload	Mismatched .jpg file accepted for DVWA and Mutillidae II	Mismatched .jpg file rejected on DVWA but still accepted on Mutillidae II
Command execution	Harmless command marker executed	The modified input was rejected on both web application

Most effective control observed:

The most effective control observed was the **DVWA Impossible security level**. It prevented both the file upload and command injection input that were accepted at the **Low security level**.

4.4 Analysis Questions

Question 1: Why is browser-only upload validation insufficient?

Answer: Validating the upload only on the browser-side isn't enough because it can easily be bypassed by modifying the request before it reaches the server. From the server end, there needs to be effective protection to validate the filename, file type and content rather than trusting client-supplied information.

Question 2: What proves that command input was treated as executable structure rather than data?

Answer: A harmless marker was attached to a valid input on both the DVWA and Mutillidae II web application, and the marker appeared in the results after submission. This shows that the application executed the marker as a backend command rather than treating it as data.

Question 3: Why is minimal proof sufficient for both weaknesses?

Answer: Since we are following the principle of least impact, the minimum proof is sufficient for both weaknesses as it already confirms the vulnerability. For that reason, there's no need for further testing that could violate the system and disclose sensitive information.

Question 4: Which control produced the greatest improvement in stronger mode?

Answer: DVWA in stronger mode produces the greatest improvement in stronger mode. For each test that was repeated with a stronger security level, the DVWA application rejected the upload and complained that the content doesn't match the extension, but Mutillidae II still uploaded the mismatched file content.

After retesting the command injection, DVWA and Mutillidae rejected the input passed through it. The command injection was also rejected.

5. REMEDIATION AND CLEANUP

5.1 Remediation Recommendations

Finding	Root Cause	Recommended Control	Priority
File Upload	Both applications accepted a filename/content mismatch. Mutillidae explicitly reported that validation was not performed.	Server-side validation for file type and content should be implemented. Mismatched content should be rejected.	
Command Injection	Uploaded files were stored using predictable/original filenames, with Mutillidae placing the file in <code>/tmp/</code> .		

5.2 Cleanup and Restoration

Test files removed: Yes - temporary files were removed

Applications/databases reset: Yes - DVWA and Mutillidae were returned to their initial state.

Final lab state: Yes - Lab environment verified clean and returned to the expected baseline state.

6. CONCLUSION

This lab was completed successfully and provided a practical understanding of how to prepare, verify, and document a controlled cybersecurity testing environment. I confirmed that the required applications were accessible and within scope, carried out the planned activities, and recorded the relevant evidence throughout the process.

At the end of the lab, I reviewed the environment, removed temporary test files and artefacts, and returned the applications and system to their expected baseline state. Overall, the exercise helped reinforce the importance of proper preparation, documentation, and cleanup when conducting security testing.

APPENDIX A — EVIDENCE REGISTER

Evidence ID	Application	Test/Action	Evidence Description
E-001	Both	Marker preparation	Marker file properties, MIME types, sizes and SHA-256 hashes
E-002	DVWA	Normal upload	Normal JPG upload result
E-003	DVWA	Retrieval	Successful browser retrieval of uploaded JPG
E-004	DVWA	Mismatch preparation	Mismatched JPG marker properties
E-005	DVWA	Mismatch upload	DVWA response to mismatched file
E-006	DVWA	Mismatch retrieval	Browser response when retrieving mismatched file
E-007	DVWA	Command baseline	Normal ping/command response
E-008	DVWA	Command execution	Harmless command marker execution
E-009	Mutillidae II	Normal upload	Normal JPG upload response

E-010	Mutillidae II	Storage verification	Container verification of stored JPG, MIME type and hash
E-011	Mutillidae II	Mismatch upload	Mismatched file accepted; validation reported as not performed
E-012	Mutillidae II	Command baseline	Normal DNS lookup response
E-013	Mutillidae II	Command execution	Harmless command marker execution
E-014	DVWA	Security-mode change	DVWA changed from Low to Impossible
E-015	DVWA	Secure upload retest	Mismatched file rejected at Impossible
E-016	DVWA	Secure command retest	Modified command input rejected at Impossible

APPENDIX B — ACTIVITY LOG

Date/Time	Application	Activity	Tool/Method	Result/Observation	Correction/Notes
28 Aug 2026	DVWA	Prepared harmless markers	Kali terminal	Recorded MIME type, size and SHA-256	No malicious content used
28 Aug 2026	DVWA	Normal JPG upload	Firefox	Upload succeeded	Baseline established
28 Aug 2026	DVWA	Mismatch upload	Firefox	.jpg file containing plain text was accepted	Stored/retrieval behaviour recorded
28 Aug 2026	DVWA	Command baseline	Firefox	Normal localhost ping returned	Baseline established
28 Aug 2026	DVWA	Minimal command test	Firefox	Harmless marker appeared in response	No further exploitation performed
28 Aug 2026	Mutillidae II	Normal JPG upload	Firefox	File moved to /tmp/	Storage subsequently verified in container
28 Aug 2026	Mutillidae II	Mismatch upload	Firefox	File accepted; validation reported as not performed	Browser retrieval not demonstrated

28 Aug 2026	Mutillidae II	Command baseline	Firefox	DNS lookup of localhost succeeded	Baseline established
28 Aug 2026	Mutillidae II	Minimal command test	Firefox	Harmless marker appeared in response	No further exploitation performed
29 Aug 2026	DVWA	Changed security level	Firefox	Changed from Low to Impossible	Stronger-mode retest
29 Aug 2026	DVWA	Upload retest	Firefox	Mismatch rejected	Stronger validation observed
29 Aug 2026	DVWA	Command retest	Firefox	Modified input rejected as invalid IP	Stronger input validation observed